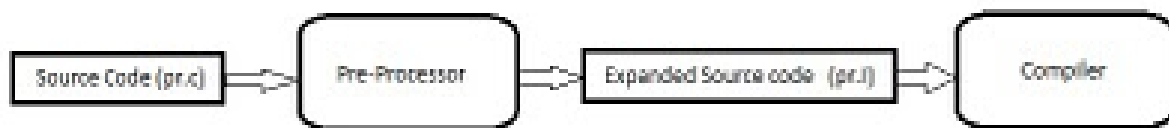


The C Preprocessor

Preprocessor directives are actually the instructions to the compiler itself. They are not translated but are operated directly by the compiler. Note that preprocessor statements begin with a #symbol, and are NOT terminated by a semicolon. Traditionally, preprocessor statements are listed at the beginning of the source file. They are also termed as macros.

The C preprocessor is a *macro processor* that is used automatically by the C compiler to transform our program before actual compilation. It is called a macro processor because it allows us to define *macros*, which are brief abbreviations for longer constructs.

In simplistic terms, a C Preprocessor is just a text substitution tool and they instruct compiler to do required pre-processing before actual compilation.



The C preprocessor provides four separate facilities that we can use:

1. **Inclusion of header files.** These are files of declarations that can be substituted into your program.
2. **Macro expansion.** We can define macros, which are abbreviations for arbitrary fragments of C code, and then the C preprocessor will replace the macros with their definitions throughout the program.
3. **Conditional compilation.** Using special preprocessing directives, we can include or exclude parts of the program according to various conditions.
4. **Line control.** If we use a program to combine or rearrange source files into an intermediate file which is then compiled, we can use line control to inform the compiler of where each source line originally came from.

Advantages of preprocessor:

1. Programs easier to develop,
2. Easier to read,
3. Easier to modify
4. C code more transportable between different machine architectures.

Macro expansion

Example:

Before preprocessing	After preprocessing
<pre>#include<stdio.h> #define TEN 10 void main() { int a = 10; if (a == TEN) { printf("The value of a is 10"); } }</pre>	<pre>#include<stdio.h> void main() { int a = 10; if (a == 10) { printf("The value of a is 10"); } }</pre>

If we run the above example, in both cases we will get the output as "The value of a is 10".

The above explained macro is called as '**Simple macro**'. There is another form of macro called 'Macros with Arguments'.

'Macros with Arguments'

Example:

Before preprocessing	After preprocessing
<pre>#include<stdio.h> #define AREA (3.14 * a * a) void main() { int ar = AREA(10); printf("Area = %f", ar); }</pre>	<pre>#include<stdio.h> void main() { int ar = 3.14 * 10 * 10; printf("Area = %f", ar); }</pre>

If we run the above example, we will get the output as "AREA = 314.000000".

In the above example, we have used macros with arguments. As we learnt, before compilation process the Source code goes through preprocessing process. During preprocessing, the program is scanned from top to bottom. Every occurrence of AREA(10) in the source code is replaced by (3.14 * 10 * 10).

Predefined Macros

ANSI C defines a number of macros. Although each one is available for use in programming, the predefined macros should not be directly modified.

Macro	Description
<code>__DATE__</code>	The current date as a character literal in "MMM DD YYYY" format
<code>__TIME__</code>	The current time as a character literal in "HH:MM:SS" format
<code>__FILE__</code>	This contains the current filename as a string literal.
<code>__LINE__</code>	This contains the current line number as a decimal constant.
<code>__STDC__</code>	Defined as 1 when the compiler complies with the ANSI standard.

Example:

```
#include <stdio.h>
```

```
main()
{
printf("File :%s\n", __FILE__ );
printf("Date :%s\n", __DATE__ );
printf("Time :%s\n", __TIME__ );
printf("Line :%d\n", __LINE__ );
}
```

When the above code in a file test.c is compiled and executed, it produces the following result:

```
File :test.c
Date :Jun 2 2012
Time :03:36:24
Line :8
```

Explanation:

The above program uses 5 macros in printf() statement. the program displays program **file name**, **system date**, **time**, **total lines** of the program.

Preprocessor Directives:

The preprocessing step happens to the C source before it is fed to the compiler. The two most common preprocessor directives are `#define` and `#include`...

1. `#define`

The `#define` directive can be used to set up symbolic replacements in the source. As with all preprocessor operations, `#define` is extremely unintelligent -- it just does textual replacement without understanding. `#define` statements are used as a crude way of establishing symbolic constants.

Example:

```
#define MAX 100
```

```
#define SEVEN_WORDS that_symbol_expands_to_all_these_words
```

Later code can use the symbols `MAX` or `SEVEN_WORDS` which will be replaced by the text to the right of each symbol in its `#define`.

SUMMARY OF `#define`

- allow the use of symbolic constants in programs
- in general, symbols are written in uppercase
- are not terminated with a semi-colon
- generally occur at the beginning of the file
- each occurrence of the symbol is replaced by its value
- makes programs readable and easy to maintain

2. `#include` [“ “—will search in current as well as standard library] [< >-- will search in standard library only]

The `"#include"` directive brings in text from different files during compilation. `#include` is a very unintelligent and unstructured -- it just pastes in the text from the given file and continues compiling. The `#include` directive is used in the `.h/.c` file convention below which is used to satisfy the various constraints necessary to get prototypes correct.

```
#include "c.h" // refers to a "user" c.h file in the originating directory
for the compile
```

```
#include <c.h> // refers to a "system" c.h file in the compiler's directory
somewhere
```

#include example**Create a file with name bca.c**

```
Int display();  
  
display();  
  
{  
    Printf("\n Coming from bca.c");  
}
```

Create another file with name call.c

```
#include<stdio.h>  
  
#include<conio.h>  
  
Void main()  
{  
    clrscr();  
    display();  
    getch();  
}
```

Output: coming from bca.c

The C preprocessor offers following operators to help us in creating macros:

1. Stringize (#)

The stringize or number-sign operator ('#'), when used within a macro definition, converts a macro parameter into a string constant. This operator may be used only in a macro that has a specified argument or parameter list.

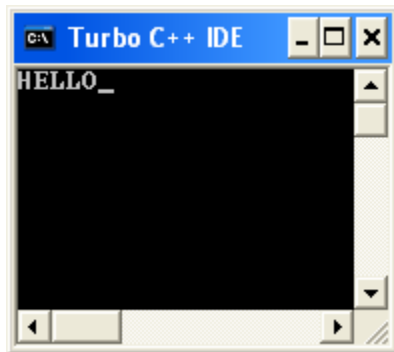
For example:

```
#include <stdio.h>

#define say(m) printf(#m)

void main()
{
    clrscr();
    say(HELLO);
    getch();
}
```

When the above code is compiled and executed, it produces the following result:



Explanation

In the above program, after conversion the statement **say (HELLO)** is treated as **printf (“HELLO”)**. It is not essential to enclose the text in quotation marks in the stringizing operator.

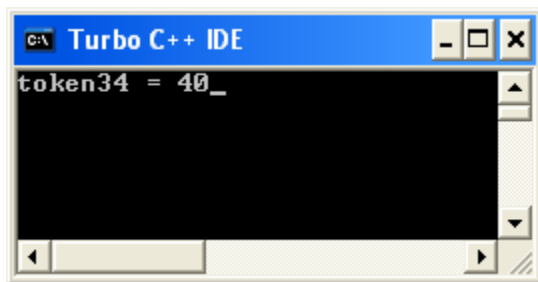
2. Token Pasting (##)

The token-pasting operator (**##**) within a macro definition combines two arguments. It permits two separate tokens in the macro definition to be joined into a single token.

For example:

```
#include <stdio.h>
#define tokenpaster(n) printf ("token" #n " = %d", token##n)
n)
void main()
{
int token34 = 40;
tokenpaster(34);
getch();
}
```

When the above code is compiled and executed, it produces the following result:



How it happened, because this example results in the following actual output from the preprocessor:

```
printf ("token34 = %d", token34);
```

This example shows the concatenation of **token##n** into token34 and here we have used both stringizing and token-pasting.

Conditional Compilation

- ✓ Six directives are available to control conditional compilation.
- ✓ They delimit blocks of program text that are compiled only if a specified condition is true.
- ✓ These directives can be nested.
- ✓ The program text within the blocks is arbitrary and may consist of preprocessor directives, C statements, and so on.
- ✓ The beginning of the block of program text is marked by one of three directives:

Some of the directives are:

1. `#if`
2. `#ifdef`
3. `#ifndef`

Optionally, an alternative block of text can be set aside with one of two directives:

1. `#else`
2. `#elif`

The end of the block or alternative block is marked by the `#endif` directive.

If the condition checked by `#if`, `#ifdef`, or `#ifndef` is **true (nonzero)**, then all lines between the matching `#else` (or `#elif`) and an `#endif` directive, if present, are ignored.

If the condition is **false (0)**, then the lines between the `#if`, `#ifdef`, or `#ifndef` and an `#else`, `#elif`, or `#endif` directive are ignored.

1. The `#if` Directive

The `#if` directive has the following syntax:

```
#if constant-expression newline
```

This directive checks whether the *constant-expression* is true (nonzero). The operand must be a constant integer expression **that does not contain any** increment (`++`), decrement (`--`), `sizeof`, pointer (`*`), address (`&`), and cast operators.

The constant expression in an `#if` directive is subject to text replacement and can contain references to identifiers defined in previous `#define` directives. The replacement occurs before the expression is evaluated. Each preprocessing token that remains after all macro replacements have occurred is in the lexical form of a token.

If an identifier used in the expression is not currently defined, the compiler treats the identifier as though it were the constant zero.

#if example

```
#include <stdio.h>

#define SWITCH 0

#if (SWITCH==1)

#define TEXT "EXAMPLE TO #IF"

#else

#define TEXT "example to else"

#endif

void main()

{

Printf(TEXT)

getch();

}
```

2. The #ifdef Directive

The `#ifdef` directive has the following syntax:

```
#ifdef identifier newline
```

This directive checks whether the identifier is currently defined. Identifiers can be defined by a `#define` directive or on the command line. If such identifiers have not been subsequently undefined, they are considered currently defined.

#ifdef example

```
#include <stdio.h>

#define LINE 1

Void main()

{
```

```

Clrscr();

#ifdef LINE

Printf("this is defined");

#else

Printf("this is not defined");

#endif

Getch();

}

```

Output : this is defined

3. The #ifndef Directive

The #ifndef directive has the following syntax:

```
#ifndef identifier newline
```

This directive checks to see if the identifier is not currently defined.

#ifndef example

```

#include <stdio.h>

#define LINE 8

Void main()

{

Clrscr();

#ifndef LINE

Printf("this is not defined");

#else

Printf("this is defined");

```

```
#endif

Getch();

}
```

Output: this is defined

4. The #else Directive

The #else directive has the following syntax:

```
#else newline
```

This directive delimits alternative source text to be compiled if the condition tested for in the corresponding #if, #ifdef, or #ifndef directive is false. An #else directive is optional.

5. The #elif Directive

The #elif directive has the following syntax:

```
#elif constant-expression newline
```

The #elif directive performs a task similar to the combined use of the else-if statements in C. This directive delimits alternative source lines to be compiled if the constant expression in the corresponding #if, #ifdef, #ifndef, or another #elif directive is false and if the additional constant expression presented in the #elif line is true. An #elif directive is optional.

6. The #endif Directive

The #endif directive has the following syntax:

```
#endif newline
```

This directive ends the scope of the #if, #ifdef, #ifndef, #else, or #elif directive.

The number of necessary #endif directives changes according to whether the elif or #else directive is used. Consider the following equivalent examples:

#if true	#if true
.	.
.	.
.	.
#elif true	.
.	#else
.	#if false
.	.
#endif	.

```

.
#endif
#endif

```

7. The defined Operator

Another way to verify that a macro is defined is to use the `defined` unary operator. The `defined` operator has one of the following forms:

```

defined name
defined (name)

```

An expression of this form evaluates to 1 if *name* is defined and to 0 if it is not.

The `defined` operator is especially useful for checking many macros with just a single use of the `#if` directive. In this way, you can check for macro definitions in one concise line without having to use many `#ifdef` or `#ifndef` directives.

For example, consider the following macro checks:

```

#ifdef macro1
printf ("Hello!\n" );
#endif

#ifndef macro2
printf ("Hello!\n" );
#endif

#ifdef macro3
printf( "Hello!\n" );
#endif

```

Another use of the `defined` operator is in a single `#if` directive to perform similar macro checks:

```

#if defined (macro1) || defined (macro2) || defined (macro3)
printf ( "Hello!\n" );
#endif

```

Note that `defined` operators can be combined in any logical expression using the C logical operators. However, `defined` can only be used in the evaluated expression of an `#if` or `#elif` preprocessor directive.